

PHÂN TÍCH KIẾN TRÚC PHẦN MỀM: COHESION, COUPLING

Nguyễn Trung Hiếu - 20226082

1. Phân tích Cohesion (Độ kết dính)

1.1. OrderService (Low Cohesion - Cần cải thiện)

- Vấn đề:** Class này đang ôm đồm quá nhiều trách nhiệm (God Class).
 - Không chỉ quản lý vòng đời đơn hàng (`createOrder`, `cancelOrder`) mà còn chứa logic tính toán phí vận chuyển (`calculateShippingFee`).
 - Trực tiếp quản lý kho hàng (trừ tồn kho sản phẩm trong `createOrder` và cộng lại trong `cancelOrder`).
 - Tự tạo đối tượng `Invoice`.
- Hệ quả:** Làm giảm tính tái sử dụng và khó viết Unit Test riêng biệt cho logic tính phí.

1.2. Cart (Low Cohesion - Anemic Model)

- Vấn đề:** `Cart` hiện tại chỉ là một cấu trúc dữ liệu (Data Class) thuần túy với các getter/setter (Anemic Domain Model). Nó thiếu các hành vi nghiệp vụ mà đáng lẽ nó phải có.

1.3. OrderController (High Cohesion)

- Đánh giá:** Tốt.
- Lý do:** Controller đóng vai trò là một lớp mỏng (thin layer), chỉ làm nhiệm vụ nhận request, validate cơ bản và ủy quyền cho Service. Đây là cách làm đúng và tiêu chuẩn trong Spring Boot.

1.4. DeliveryInfo (High Cohesion - Data)

- Đánh giá:** Tốt.
- Lý do:** Là một Embeddable class lưu trữ thông tin giao hàng.

2. Phân tích Coupling (Độ phụ thuộc)

2.1. OrderService phụ thuộc chặt vào ProductRepository (Tight Coupling)

- **Vấn đề:** `OrderService` đang gọi trực tiếp `ProductRepository` để `save` (cập nhật tồn kho).
- **Hệ quả:** Nếu logic lưu trữ Product thay đổi, hoặc cần thêm logic khi trừ kho (ví dụ: gửi cảnh báo khi sắp hết hàng), bạn phải sửa code trong `OrderService`.

2.2. PaymentService phụ thuộc chặt vào OrderRepository (Tight Coupling)

- **Vấn đề:** `PaymentService` đang trực tiếp tìm và sửa trạng thái của `Order` (`updateOrderPaid`).
- **Hệ quả:** Payment module "biết" quá nhiều về nội tại của Order module. Nếu luồng trạng thái Order thay đổi, `PaymentService` cũng bị ảnh hưởng theo.

2.3. Coupling với Logic Tính Phí (Tight Coupling)

- **Vấn đề:** Logic tính phí vận chuyển (`calculateShippingFee`) được hardcode ngay trong `OrderService`.
 - **Hệ quả:** Nếu muốn thay đổi chiến lược tính phí (ví dụ: thêm hãng vận chuyển mới, thay đổi bảng giá), phải sửa và deploy lại core service này.
-

3. Tổng kết & Đánh giá

- **Hiện trạng:** Code chạy được nhưng về mặt kiến trúc (Architecture) đang thiên về hướng **Procedural Programming** (lập trình thủ tục) được bao bọc trong các class Service, thay vì **Object-Oriented Programming** (hướng đối tượng) thực sự như Diagram gợi ý.
- **Tự đánh giá:**
 - **Cohesion:** Trung bình - Thấp (Logic nghiệp vụ bị trộn lẫn trong Service).
 - **Coupling:** Cao (Các Service gọi chéo Repository của nhau).

Kết luận: Cần thực hiện Refactor để đạt được Clean Code và đồng bộ với thiết kế ban đầu.